

ARQUITECTURA

■ CyberAgent LLM — Arquitectura del Ecosistema

- > **Documento maestro.** Mapa completo del sistema: módulos, submódulos, relaciones,
- > guía de configuración y métodos de arreglo. Es un **ecosistema vivo** que une dos
- > grandes sistemas — el **cerebro** (CyberAgent, agente IA local) y el ****módulo de**
- > **seguridad**** (APiComuni v2: cámaras, Telegram, Home Assistant, disuasión).
- >
- > **Versión:** 1.0 · **Fuente del backlog:** TASKBOARD.md (grupos SEC + A..BA).
- > **Estilo:** CyberAgent LLM. **Cerebro:** modelo local. **Secretos:** vault cifrado.

Índice

1. [Visión general](#1-visión-general)
2. [Principios de diseño](#2-principios-de-diseño)
3. [Mapa de módulos](#3-mapa-de-módulos)
4. [El cerebro (CyberAgent core)](#4-el-cerebro-cyberagent-core)
5. [Interfaces (GUI · Web · iOS · Telegram)](#5-interfaces)
6. [Módulo de seguridad](#6-módulo-de-seguridad)
7. [Visión por computador (cámaras)](#7-visión-por-computador)
8. [Reconocimiento de mascotas + patrones](#8-reconocimiento-de-mascotas--patrones)
9. [Disuasión + actuadores](#9-disuasión--actuadores)
10. [Comunicaciones (Telegram pro)](#10-comunicaciones-telegram-pro)
11. [Entrenamiento (QLoRA) + feedback](#11-entrenamiento-qlora--feedback)
12. [Coordinación de recursos (GPU/CPU/VRAM)](#12-coordinación-de-recursos)
13. [Almacenamiento (SD 1.8 TB)](#13-almacenamiento)
14. [Secretos + seguridad del sistema](#14-secretos--seguridad-del-sistema)
15. [Supervisión + auto-recuperación](#15-supervisión--auto-recuperación)
16. [Mapa de relaciones entre módulos](#16-mapa-de-relaciones)
17. [Guía de configuración](#17-guía-de-configuración)
18. [Métodos de arreglo (troubleshooting)](#18-métodos-de-arreglo)
19. [Glosario](#19-glosario)

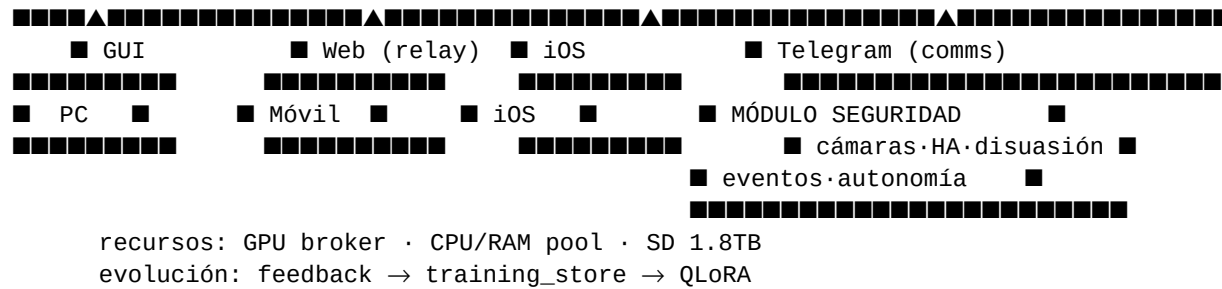
1. Visión general

CyberAgent LLM es un agente de IA **local-first** que corre en el PC del usuario y se expone por GUI de escritorio, web móvil (vía relay Cloud Run) e iOS. Sobre ese cerebro se integra un **módulo de seguridad domótica** (cámaras, Home Assistant, Telegram) que NO usa una IA externa: usa el **mismo cerebro local** + Mistral nube para visión instantánea.

```
■■■■■■■■■■■■■■■■■■■■ CERE BRO (CyberAgent core) ■■■■■■■■■■■■■■■■■■■■
```

```
■ Modelos: cyberagent-24b (agente) · Codestral (código) · Mistral ■
```

```
■ Modo Claude · Tools (120+) · Aprobaciones · RAG · Persistencia ■
```



2. Principios de diseño

| Principio | Qué significa |

|-----|-----|

| **Local-first** | El cerebro vive en el PC; la nube (Mistral) solo bajo demanda o por latencia. |

| **Degradación elegante** | Si algo no está (BT, GPU, modelo), cae al siguiente recurso, nunca se rompe. |

| **El usuario tiene prioridad** | La seguridad nunca bloquea al usuario; degrada a nube/CPU. |

| **Todo gateado** | El módulo de seguridad entra desactivado (SECURITY_ENABLED) salvo lo esencial. |

| **Secretos centralizados** | Todo pasa por el vault cifrado; revelar exige 2FA. |

| **Ecosistema vivo** | El uso genera feedback → entrena los modelos → el sistema mejora solo. |

| **Auto-recuperación** | Supervisor + watchdog: cada parte se vigila y se cura sola. |

| **Poder del agente local** | Un agente local con 120+ tools, acceso total al sistema, Docker, HA, cámaras, disuasión y modos (Claude/código/imagen) es ENORMEMENTE potente y privado. Por eso se construye tan bien y tan robusto: cada capacidad nueva multiplica lo que puede hacer por el usuario, sin depender de la nube. Es la razón de ser del proyecto. |

3. Mapa de módulos

| Módulo | Carpeta | Estado | Responsabilidad |

|-----|-----|-----|-----|

| **Core / Agente** | app/, app/api/ | ■ activo | Razonamiento, tools, modelos, persistencia |

| **Interfaces web** | apps/web/ | ■ activo | PWA móvil, relay |

| **GUI escritorio** | app/widgets/ | ■ activo | App nativa PC |

| **iOS** | ios/ | ■ siguiente | App nativa iOS |

| **Comunicaciones** | app/comms/ | ■ parcial | Telegram, notificaciones centralizadas |

| **Seguridad** | app/security/ | ■ esqueleto | Cámaras, eventos, HA, disuasión |

| **Secretos** | app/secrets_vault.py | ■ activo | Vault cifrado |

| **Docker** | app/docker_tools.py | ■ activo | Gestión de contenedores |

| **Entrenamiento** | app/training/ | ■ planificado | QLoRA, feedback, umbrales |

| **Recursos** | app/compute/ | ■ planificado | GPU broker, CPU/RAM pool, modo juego |

| **Almacenamiento** | app/storage/ | ■ planificado | SD, retención video, datasets |

| **Supervisor** | app/supervisor.py, watchdog.py | ■ activo | Auto-recuperación |

4. El cerebro (CyberAgent core)

- **Modelos locales** (Ollama): cyberagent - 24b (Mistral Small abliterado Q3_K_M, agente general ~45 tok/s), cyberagent - codestral (código). **Nube**: Mistral (Small/Medium/Large/Codestral/Pixtral). **Modo Claude** (CLI con skip-permissions).
- **Routing** (app/model_router.py, app/brain.py): elige modelo; modo relevo 24B+Codestral; auto/fused.
- **Tools** (app/tools.py, app/tools_ext.py): 120+ herramientas; tool_router.py selecciona el subconjunto relevante.
- **Aprobaciones**: DANGEROUS_TOOLS → el usuario confirma (la autonomía de seguridad mapea a esto).
- **Persistencia**: SQLite (chats, carpetas, archivos, favoritos) + RAG vectorial.
- **Submódulos clave**: agent_runner.py (bucle del agente), ollama_client.py (inferencia local), vision.py (visión), mistral_studio.py (FLUX/nube).

5. Interfaces

Interfaz	Tecnología	Notas
GUI escritorio	PySide6	Tray, ventana, modos
Web móvil	PWA + relay Cloud Run	WebSocket por session_id; contexto por sesión (multi-usuario)
iOS	SwiftUI	■ a cablear (incluye vista Seguridad)
Telegram	Bot API	Canal de comunicaciones centralizado (no solo seguridad)

6. Módulo de seguridad

app/security/ — APiComuni v2. **Gateado por** SECURITY_ENABLED (todo desactivado salvo notificación Telegram). Submódulos:
brain_bridge (IA = nuestro agente), camera, motion, events, autonomy (→ aprobaciones), actions, pets, zones, deterrence, actuators/.
La IA de visión usa **Mistral nube (Pixtral)** para reacción instantánea.

7. Visión por computador

Pipeline por niveles (ver §12):

1. **Movimiento (CPU, 0 VRAM)** — OpenCV, vigila sin parar.
2. **Triage** — VLM local ligero (Moondream2) si GPU libre, o Pixtral nube.
3. **Análisis profundo** — Pixtral nube (descripción policial, amenaza, decisión).
Cámaras **exterior** (intrusión, descripción policial, disuasión) e **interior** (gatos, peligros domésticos). Zonas dibujables (warning/segura, prevalece mayor riesgo).

8. Reconocimiento de mascotas + patrones

- **Detección** → caja del animal. **Re-ID** → embedding + pelaje + proporciones → cuál gato (few-shot desde fotos). **Tracking** → trayectoria.

- **Patrones (auto-supervisado)**: predice movimiento → comprueba → feedback +/- → aprende. Genérico por especie + por individuo. Anomalía = fuera de patrón.

9. Disuasión + actuadores

Actuadores abstractos (app/security/actuators/): la IA pide intención/nivel (1 presencia → 2 audio → 3 narración en vivo → 4 luz → 5 escalar), la capa elige el actuador disponible (degradación BT→sistema→nada). Hoy: **altavoz BT potente** (sonidos por escenario + TTS narrando lo que ve). Futuro: luces/enchufes inteligentes (añadibles a HA desde el menú de la cámara). Semáforo rojo/ámbar/verde: el agente cablea y testea; **verde = funciona y se puede emitir prueba real**.

10. Comunicaciones (Telegram pro)

app/comms/ — **central, no solo seguridad** (respuestas de agentes, errores, amenazas). **Supergrupo FORO con Topics** → hilos por importancia (Urgente/Seguridad/Notif/Gatos/Periódico/Sistema), 1 bot. Severidades (crítica→sonido+pin / periódica→silencio→digest). **Panel inline por mensaje** (Confirmar/Ignorar/Ver cámara/Silenciar/Disuadir). Comandos + chat con el agente + reacciones ■/■ (→ entrenamiento).

11. Entrenamiento (QLoRA) + feedback

- **Feedback** (training_store): "¿útil?" + "¿razonamiento correcto?" + correcciones + aprobaciones + falsos pos/neg de seguridad. Etiquetado por modelo.
- **Umbral por modelo** (24b ~1500, Codestral ~1000, visión ~500). Al alcanzarlo → aviso multicanal + badge en Ajustes → Entrenamiento.
- **Menú** (Ajustes): progreso por modelo, botón Entrenar (preflight), progreso en vivo, A/B + promover/rollback. **Solo en PC** (VRAM/presencia/seguridad).
- **Motor**: local si cabe / RunPod A100 si no. Merge adapter → nuevo modelo Ollama.

12. Coordinación de recursos

16 GB VRAM compartida entre usuario y seguridad:

- Vigilar = **CPU** (0 VRAM). GPU/nube solo con movimiento.
- GPU libre → VLM local; GPU ocupada → Pixtral **nube** (instantáneo).
- **El usuario tiene prioridad**; la seguridad degrada a nube/CPU.
- 24B en Q3 (~11GB) + Moondream2 (~2.5GB) **caben juntos**.
- **Modo juego**: libera el 24B, seguridad a nube, pausa entrenamiento.
- **CPU/RAM (64GB)**: movimiento, transcripción, embeddings, caché de frames.

13. Almacenamiento

SD 1.8 TB: /models /datasets /videos /backups. Video **H.265, retención legal 15 días** (auto-borrado), índice en DB. Datasets versionados comprimidos. Cuotas + alertas de espacio. Backups del vault/DB.

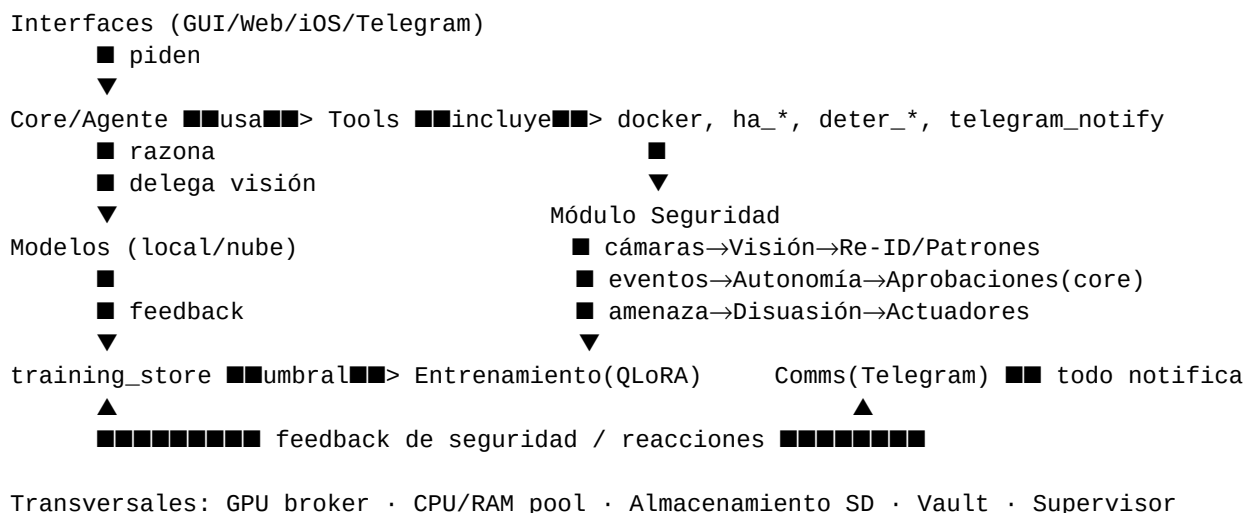
14. Secretos + seguridad del sistema

app/secrets_vault.py — **Fernet cifrado** (data/vault.enc + .vault_key, gitignored). **2 claves Mistral**: MISTRAL_API_KEY (CyberAgent) y SEC_MISTRAL_API_KEY (seguridad). Todo lo de APiComuni con prefijo SEC_. Revelar valores en la web exige **TOTP** (authenticator). Reutilizado por todo el sistema.

15. Supervisión + auto-recuperación

app/supervisor.py (in-process, 4 servicios: persistencia, Ollama, conexión, watchdog) + watchdog.py (externo, reinicia la app si se cuelga). El módulo de seguridad/comms se añade como servicios gateados. **4 capas de auto-cura**: conector → supervisor → watchdog externo → supervisión mutua.

16. Mapa de relaciones



Relaciones clave:

- **Seguridad** → **Core**: la autonomía de seguridad usa el sistema de **aprobaciones** del core.
- **Seguridad** → **Comms**: toda alerta sale por el módulo de comunicaciones central.
- **Todo** → **training_store**: feedback de cualquier módulo alimenta el entrenamiento.
- **Entrenamiento** ↔ **GPU broker**: al entrenar, la seguridad degrada a nube.
- **Actuadores** → **HA / Audio**: disuasión vía Home Assistant o altavoz BT.

17. Guía de configuración

1. **Secretos** (una vez): se importan al vault desde E:\APiComuni\...\env (prefijo SEC_) + el .env de CyberAgent. Revelar/editar en Ajustes → Secretos (2FA).
2. **Telegram**: SEC_TELEGRAM_BOT_TOKEN + SEC_TELEGRAM_CHAT_ID (ya en el vault). Notificación activa de inmediato. Para Topics: crear supergrupo foro (tarea AN-01).

3. **Activar seguridad:** CYBERAGENT_SECURITY_ENABLED=1 (cuando los submódulos estén verdes).
4. **Cámaras:** añadir en el dashboard (tipo exterior/interior, RTSP/HA, ubicación, zonas).
5. **Actuadores:** en la vista de cámara, añadir altavoz BT / dispositivo HA, describir comportamiento, "Cablear" → el agente testea → **verde** → "Probar".
6. **Entrenamiento:** Ajustes → Entrenamiento; cuando un modelo llegue al umbral, "Entrenar" (PC).
7. **Almacenamiento:** configurar ruta base en la SD; retención de video 15 días.

18. Métodos de arreglo (troubleshooting)

Síntoma	Causa probable	Arreglo
"PC no conectado" en la web	Conector fijado a revisión vieja tras redeploy	El supervisor lo detecta y reconecta (§15); si no, reiniciar la app del PC
Respuestas lentas / 4 min	Modelo equivocado cargado (CYBERAGENT_FAST_MODEL)	Verificar que es cyberagent - 24b; reiniciar app
: 8765 no enlaza	pythonw sin stdout / puerto ocupado	Fix de devnull aplicado; el supervisor reenlaza
Ollama colgado	VRAM saturada / 2 modelos grandes	OLLAMA_MAX_LOADED_MODELS=1; el supervisor reinicia Ollama
Telegram no envía	Token/chat no en el vault	Revisar Ajustes → Secretos (SEC_TELEGRAM_*)
Actuador en ROJO	Sin cablear o desconectado (BT caído)	"Cablear" de nuevo; degrada a altavoz del sistema
Cámara sin stream	RTSP/HA caído	Revisar SEC_HA_* / proxy de stream; ver §7
Docker tool falla	Daemon parado	Arrancar Docker Desktop
Entrenamiento no arranca	Usuario no presente / sin VRAM	Solo en PC con presencia; liberar VRAM
Vault no revela	TOTP incorrecto	Código del authenticator (RELAY_TOTP_SECRET)

Diagnóstico general: GET /api/health (los 4+ servicios) · logs en agent.log · docs/ por módulo · TASKBOARD.md para el estado de cada pieza.

19. Glosario

- **Cerebro:** el agente IA local (cyberagent-24b + tools).
- **Actuador:** dispositivo de disuasión abstracto (altavoz, luz...).
- **Re-ID:** identificar CUÁL individuo (gato) por embedding.
- **training_store:** almacén de feedback que alimenta el QLoRA.
- **GPU broker:** árbitro que da prioridad de GPU al usuario.
- **Topics:** hilos separados dentro de un supergrupo de Telegram.
- **Degradación elegante:** caer al siguiente recurso disponible sin romperse.
- **Verde:** un actuador/servicio cableado y verificado funcionando.

- > **Siguiente paso del proceso:** esperar feedback del usuario, apuntarlo en
- > TASKBOARD.md, y continuar. Este documento se actualiza con cada cambio mayor
- > de arquitectura.